ICS 33 Fall 2026
Notes and Examples: Classes and Objects

A brief review of classes in Python

In your previous studies of Python, we expect that you will already have learned about something called a *class*, which is a way to define a new type of object, specifying both the information that objects of the new type will store, as well as the details of how you'll be able to interact with them. While Python has many types built into both the language and its standard library, there is naturally a limit to what can be provided out of the box — the less common the problem you're solving is, the smaller the chance you'll find a type already within Python that solves it — so we'll need a way to specify our own when the need arises. Writing classes is how we do that in Python.

This is not a topic we expect to be entirely new territory, but there's plenty of fertile ground for exploration, because we're going to need to understand classes in a lot more depth than you've probably seen previously. Much of what we'll be discussing over the next few weeks will revolve around enriching our understanding of classes, as they form one of the pillars of Python's design. Before moving any further with this set of notes, though, you'll want to first be sure that you're up to speed on the notes from my ICS 32 (or H32) course, which describe the basics of how to write a class and its methods, what the attributes of an object are, how to specify the initialization of a class' objects using the `__init__` method, and how to call an object's methods. Your best bet is to read those notes first, then proceed from there, as I'll be assuming full familiarity with the specific way that I've presented this topic in past coursework — which should track pretty well with your understanding, even if you didn't happen to take these courses with me — then building on top of that familiarity.

- Classes (from ICS H32)

What's inside of objects and their classes?

As we do with many programming language features when we first learn about them, we can use classes and their objects fairly effectively with only a limited understanding of how they actually work. Skipping many of the details allows us to learn new topics by applying them to problems without feeling overwhelmed by complexity, but there comes a time when we need to go back and fill in the details we've skipped. When it comes to classes, now is the time, because those details will form the backbone of Python features we've yet to learn.

Like we did when we explored modules, namespaces, and importation in depth, we can start by using the Python shell to experiment with the inner workings of classes and their objects. Let's begin by building a tiny class.

```
>>> class Thing:
...     pass
...
```

When we were learning about modules previously, a collision in terminology arose, though you might not have noticed it at the time.

- You may have learned before that objects in Python have *attributes*, which is where their information is stored.
- Recently, we learned that modules in Python also contain a collection of *attributes*, each of which has a value. It's more common for those attributes to be functions or classes than, say, integers or strings, but they can actually store any type of Python value you'd like. (A global variable in a Python module is really just an attribute of that module with some value stored in it, ultimately.)

There's one more interesting tidbit: When you try to call a non-existent method on an object of some class, the exception that's raised is known as an `AttributeError`, which suggests that maybe there's more going on there than you were first taught: Methods are attributes, too.

```
>>> s = 'Boo'
>>> s.cant_do_this()
    Traceback (most recent call last):
      ...
    AttributeError: 'str' object has no attribute 'cant_do_this'
```

If Python's modules have their attributes stored in a dictionary that we can access via an attribute named `__dict__`, perhaps it's reasonable to imagine that objects do, too.

```
>>> t = Thing()
>>> type(t.__dict__)
    <class 'dict'>
>>> t.__dict__
    {}
```

When we created a `Thing` object and asked it for its `__dict__`, we got back an empty one. And, if you think about it, that seems pretty reasonable. Since our class is empty, it has no methods, which means that it has no `__init__` method, so nothing ever stored a value in any of our object's attributes. Consequently, its dictionary is empty. So, we strongly suspect that an object's dictionary is where its attributes are stored. To confirm those suspicions, let's see what happens to the dictionary when we store values in a couple of attributes within the object.

```
>>> t.name = 'Boo'
>>> t.age = 13
>>> t.__dict__
    {'name': 'Boo', 'age': 13}      # Suspicion confirmed.
```

On the flip side of the same coin, let's see what happens when we modify the dictionary directly.

```
>>> t.__dict__['age'] = 16
>>> t.age
    16                              # Updating an attribute's value updates the object.
>>> t.__dict__['color'] = 'tan'
>>> t.color
    'tan'                           # Assigning to a new attribute's adds an attribute to the object.
```

If methods are attributes, then we would expect to see them appear in an object's dictionary, as well. To test that theory, let's write a class with a single method.

```
>>> class Squarer:
...     def square(self, n):
...         return n * n
...
>>> s = Squarer()
>>> s.square(3)
    9               # We can call square on a Squarer.
>>> s.__dict__
    {}              # But where's the square method?
```

Recall what you saw in the ICS H32 notes that I linked previously, though. There are two ways to call methods in Python:

- By specifying the target object, the method, and all of the method's parameters except `self`.

```
>>> s.square(3)
    9
```

- By specifying the class, the method, and all of the method's parameters including `self`.

```
>>> Squarer.square(s, 3)
    9
```

While we tend to prefer the first of these syntaxes when we have no reason to choose one over the other — since it's the notation that's most straightforward for a human reader, conveying nicely the idea that an object is being asked to do a job — the truth is that both of these call the same function in the same way. And if we can call `Squarer.square` by passing `s` as an argument, then it seems reasonable to imagine that perhaps the `square` method isn't associated with each object at all. Perhaps it's associated, instead, with the class. (This is actually an idea with a lot of merit, because if you had millions of objects of the same type, all of which contained the same ten methods, why store an extra ten references in each object's dictionary, when we could instead store them once in the class and be done with it?)

So, if our theory is that a class has its own dictionary, and that its dictionary stores its methods, let's test that theory.

```
>>> type(Squarer.__dict__)
    <class 'mappingproxy'>
>>> Squarer.__dict__
    mappingproxy({'__module__': '__main__',
    'square': <function Squarer.square at 0x000002F275F4B640>,
    '__dict__': <attribute '__dict__' of 'Squarer' objects>,
    '__weakref__': <attribute '__weakref__' of 'Squarer' objects>,
    '__doc__': None})
```

The results here are a little more mixed: We see that `Squarer` has a `__dict__` attribute, that it contains `square`, and that `square` is a function, all of which are in line with our expectations. But we also see some other things that are a little less obvious.

- `Squarer.__dict__` is not technically a dictionary, but instead something called a `mappingproxy`, though it still feels like a dictionary, in the sense that it looks like it's associating strings with values.
- `Squarer.__dict__` contains a few other things that we didn't write in the class ourselves, like `__module__` and `__doc__`.

Let's take these issues in turn. If a `mappingproxy` looks like a dictionary, can we treat it like one?

```
>>> Squarer.__dict__['square']
    <function Squarer.square at 0x000002F275F4B640>
>>> Squarer.__dict__['square'](s, 3)
    9
>>> Squarer.__dict__['square'] = 13
    Traceback (most recent call last):
      ...
    TypeError: 'mappingproxy' object does not support item assignment
```

It looks like the answer is that we can read from it like a dictionary, but not write into it (i.e., change the value associated with a key, add a new key that isn't already there) like we could a dictionary. But, for our purposes, we can think of it as a dictionary — we won't generally want to write directly to the dictionaries belonging to objects, classes, or modules, anyway — and move on to greener pastures.

What about the other question we had? What are the other things in `Squarer`'s dictionary?

- `__module__` specifies the name of the module in which the class was defined. Since we defined `Squarer` in the Python shell, the module's name is listed as `__main__`.
- `__doc__` is where the class' docstring is stored. Had we written one, we would see it there, instead of seeing `None`.
- `__dict__` and `__weakref__` involve longer and less-important stories that we'll defer for another day.

Summarizing what we've seen so far

What have we seen in our exploration of the internals of objects and classes so far?

- An object contains a dictionary where its attributes are stored. We can access it via `__dict__` on the object (e.g., if `x` is a variable in which an object is stored, `x.__dict__` will give us that dictionary).
- A class also contains a dictionary where its attributes are stored, though it differs in the details of what's in it. We can access it via `__dict__` on the class (e.g., if we have a class `Person`, we can access its dictionary with `Person.__dict__`).

However, there's still a gray area to be explored here. If every object has a class, then how does Python decide what's stored in the object's dictionary and what's stored in its class' dictionary instead? When we interact with an object, how does Python decide whether to look for an attribute in the dictionary belonging to the object or its class? From a design perspective, which attributes do we want to belong to the class, and which do we want to belong to each object?

Getting to the bottom of those questions begins by knowing a bit more about the underlying mechanisms that Python uses to find attributes within objects and their classes.

Accessing the attributes of objects and classes

Let's begin again with a fresh Python shell and further explore objects and their classes. Even in prior coursework, you will have seen the techniques below, but we're now interested in the details that make them work.

```
>>> class Person:
...     def __init__(self, name, age):
...         self.name = name
...         self.age = age
...     def describe(self):
...         return f'{self.name}, age {self.age}'
...
>>> p1 = Person('Alex', 47)
>>> p2 = Person('Boo', 13)
>>> p1.name
    'Alex'
>>> p2.age
    13
>>> p1.describe()
    'Alex, age 47'
>>> Person.describe(p2)
    'Boo, age 13'
```

Now that we have a class and a couple of objects to work with, we can inspect their internals a bit.

```
>>> Person.__dict__
    mappingproxy({'__module__': '__main__',
    '__init__': <function Person.__init__ at 0x0000026E81E7D5A0>,
    'describe': <function Person.describe at 0x0000026E81E7D7E0>,
    '__dict__': <attribute '__dict__' of 'Person' objects>,
    '__weakref__': <attribute '__weakref__' of 'Person' objects>,
    '__doc__': None, '__annotations__': {}})
>>> p1.__dict__
    {'name': 'Alex', 'age': 47}
>>> p2.__dict__
    {'name': 'Boo', 'age': 13}
```

Ignoring the *dunder* attributes of the `Person` class (i.e., those whose names begin and end with double underscores, which are sometimes referred to as "dunders"), we can theorize that Python stores methods in a class and other data in its objects

But is that theory really true? We've previously seen that a `def` statement in a module builds a function and stores it in an attribute of that module, and it looks like classes behave similarly. But we've also seen that a module's attributes can store values that aren't functions. Can we do that in classes? Suppose we wrote this short module.

person.py *(click here for a commented version)*

```
class Person:
    MAX_NAME_LENGTH = 30

    def __init__(self, name, age):
        self.name = name[:self.MAX_NAME_LENGTH]
        self.age = max(0, age)

    def describe(self):
        return f'{self.name}, age {self.age}'
```

Let's execute this module in the Python shell and interact with it.

```
>>> p1 = Person('Boo', 13)
>>> p1.name
    'Boo'
>>> p1.age
    13
>>> p1.MAX_NAME_LENGTH
    30
```

So far, our theory seems to be holding up fine: Since `MAX_NAME_LENGTH` is not a function, we can access it within the object — both in the Python shell where we wrote `p1.MAX_NAME_LENGTH` and in the `__init__` method where we wrote `self.MAX_NAME_LENGTH`. Therefore, we would expect to find `MAX_NAME_LENGTH` in `p1.__dict__`.

```
>>> p1.__dict__
    {'name': 'Boo', 'age': 13}       # <-- Where's MAX_NAME_LENGTH?
>>> Person.__dict__
    mappingproxy({'__module__': '__main__', 'MAX_NAME_LENGTH': 30,     # <-- There it is!
    '__init__': <function Person.__init__ at 0x00000184C1EFC8B0>,
    'describe': <function Person.describe at 0x00000184C1EFC9D0>,
    '__dict__': <attribute '__dict__' of 'Person' objects>,
    '__weakref__': <attribute '__weakref__' of 'Person' objects>,
    '__doc__': None, '__annotations__': {}})
```

Given this, we'll need to think about our assumptions a little more carefully. What's really going on here? There are four things we need to understand, for the time being.

1. When we define any value within a class, whether it's a `def` statement or an assignment, we're specifying a *class attribute*. Our `Person` class has three attributes (aside from those created automatically behind the scenes): the `__init__` method, the `describe` method, and the `MAX_NAME_LENGTH` value.
2. When we store any value within an object, such as when we wrote `self.name`, we're specifying an *object attribute*.
3. When we access an attribute of an object, Python first checks whether that attribute is defined within the object. If so, that's what we get. If not, Python then checks whether that attribute is defined within the class. If so, that's what we get. If not, an `AttributeError` is raised.
4. When we access an attribute of a class, Python only checks whether the class has that attribute. If so, that's what we get. If not, an `AttributeError` is raised.

Let's test our understanding in the Python shell.

```
>>> p1.name
    'Boo'        # This works, because p1 has a name attribute.
>>> p1.MAX_NAME_LENGTH
    30           # This works, because MAX_NAME_LENGTH is found in the class,
                 # even though it's not found in the object.
>>> Person.MAX_NAME_LENGTH
    30           # This works, because MAX_NAME_LENGTH is found in the class,
                 # which is the only place where Python will look.
>>> Person.name
    Traceback (most recent call last):
      ...
    AttributeError: type object 'Person' has no attribute 'name'
                 # The Person class has no name attribute.  If we want a
                 # person's name, we have to specify which person's name we want.
```

Our understanding of Python's attribute lookup rules also tells us what should happen when the same attribute appears in a class and one of its objects, but let's verify our understanding.

```
>>> class Xyz:
...     value = 11
...     def __init__(self):
...         self.value = 17
...
>>> x = Xyz()
>>> x.__dict__['value']
    17           # The object has a value attribute.
>>> Xyz.__dict__['value']
    11           # The class also has a value attribute.
>>> x.value
    17           # When objects have attributes, they "win".
>>> Xyz.value
    11           # When we look up attributes in classes, we get the class' attributes,
                 # even if objects of that class have their own attributes with the same name.
```

So, now that we've seen the details of how class attributes and object attributes work differently, we should consider how we might summarize our understanding. It seems that the rules boil down to this.

- Objects can store attributes, which are the values that differentiate one object of a class from another. Different people have different names, different strings contain different characters, and so on.
- Classes can store attributes, which are values that are equally applicable to all objects of the same class, which is to say that they describe the type rather than each object of that type.
- Methods are stored as class attributes, but we call them on individual objects, which means that a method is a way to ask an object to do some job. The `self` parameter gives us a way to interact with the object that a method was called on. Storing methods as class attributes is vitally important: It's what makes objects of a class all behave similarly.

But there's another question worth considering. Objects store values in their attributes, and they have methods that let us interact with them. Classes also store values in their attributes, so doesn't it also make sense that there be a method that we can call on a class as a whole, rather than calling it on an individual object? In other words, if a class can contain information that doesn't require objects to access, aren't there jobs that a class could do that also don't require an object? If so, what mechanism would give us the ability to write a method with no `self` parameter?

Static methods and class methods

Static methods and class methods

When a class attribute stores a value such as an integer, its value is meant to describe something about the class as a whole, rather than an individual object of that class. For example, one of the `Person` classes we saw previously included a `MAX_NAME_LENGTH` attribute in the class, because it describes something about the class in its entirety, since all `Person`s have the same maximum, rather than each having a separate one. And, in fact, there's a maximum length whether there are any `Person` objects or not.

So, certainly, one reason we might want to use class attributes is to store what are essentially class-scoped named constants. But it's not necessarily going to be the case that class attributes have values that are constants; they just need to have values that are meaningful class-wide, rather than being separately meaningful for each object of that class.

Suppose, for example, that we're implementing a class called `Widget`. It's not particularly important what problem `Widget` solves, except that one thing we'd like to know is how many `Widget`s have been created since our program started running. Somewhere we'll need an integer value that is the answer to this question, and we'll need to update it accordingly when that answer changes. But there's only one answer to this question — How many `Widgets` are there? — no matter how many `Widget` objects have been created, and even if no `Widget` objects have been created. Nonetheless, it's absolutely and directly related to the `Widget` class. So, a class attribute is a natural place to store it.

We'd want it to be possible to ask what the value is, but not for code outside of the class to change it, so it seems wise that we mark this class attribute as protected (i.e., prepend an underscore to its name). But we'll then need a way for code outside of the class to obtain its value. For this purpose, we'll want a *static method*, which is a method that differs in two ways from the methods you've seen previously.

- It's called on the class as a whole, rather than on an object of that class.
- Since it's not called on an object of that class, it has no `self` parameter, and one is not "filled in" automatically when you call it.

To tell Python that we want to treat a method specially in this way, we mark it with `@staticmethod` on the line above it. These kinds of markings, starting with the symbol `@`, are called *decorators*, and we'll see them in more detail later this quarter, but you can think of them, for now, as being a way to tell Python to treat something differently than it might normally be treated. Normally, methods have a `self` parameter and we call them on an object; with the `@staticmethod` decorator above them, they become methods we call on the class instead.

widget.py *(click here for a commented version)*

```
class Widget:
    _count = 0

    def __init__(self, id):
        self._id = id
        Widget._count += 1

    def id(self):
        return self._id

    @staticmethod
    def widget_count():
        return Widget._count
```

After executing that module, we can interact with it, so we can see how using static methods is different from using non-static ones.

```
>>> Widget.widget_count()
    0            # Initially, no Widgets have been created.
>>> w1 = Widget(13)
>>> w2 = Widget(18)
>>> w1.id(), w2.id()
    (13, 18)     # Individual Widgets have their own IDs.
>>> Widget.widget_count()
    2            # Creating Widget objects adds to the count.
>>> w1.widget_count()
    2            # We can call static methods on Widget objects notationally, but
                 # the usual mechanics of w1 becoming a self parameter are
                 # not applied.
>>> Widget.id()
    Traceback (most recent call last):
      ...
    TypeError: Widget.id() missing 1 required positional argument: 'self'
                 # We can't call non-static methods on the entire class.  If we aren't
                 # specific about which Widget we're asking about, it doesn't
                 # make sense to ask for an ID, since each Widget has its own.
```

Class methods

There is another variation on this theme that's worth taking note of. A *class method* is similar to a static method, in the sense that it applies to the class as a whole, rather than to individual objects of that class. How a class method is different, though, is that the class becomes a parameter to that method, similar to how normal methods transform the target object into a `self` parameter, which is useful if we'll need to use the class for something.

One example where class methods are useful is for creating what are sometimes called *factory methods*, which are methods whose job is to create objects of a type, but that have names that make clearer the way that job is being done, which can be especially useful if there's more than one of them in the same class, though can still make for a more readable syntax even if there's only one of them.

point.py *(click here for a commented version)*

```
class Point:
    @classmethod
    def from_cartesian(cls, x, y):
        return cls(x, y)

    def __init__(self, x, y):
        self._x = x
        self._y = y

    def x(self):
        return self._x

    def y(self):
        return self._y
```

Given that class, we can create objects using the somewhat more humane and readable name `Point.from_cartesian`, making clear not just that we're creating a `Point`, but that we're creating it from Cartesian coordinates. (If we additionally had a way to create them from some other coordinate system, like polar coordinates, then this technique would shine more brightly.)

```
>>> p = Point.from_cartesian(11, 18)
>>> p.x(), p.y()
    (11, 18)
```